

# SP-4 Phase F.2 — Per-clone URL routing (detailed design)

**Spec date:** 2026-05-13 **Parent SP:** docs/superpowers/specs/2026-05-12-sp4-multi-provider-redesign.md **Prerequisite:** Phase F.1 complete (HEAD b54e7e63 and later). **Status:** detailed-design locked. Execution is a cross-module refactor across every tracker plugin (6 today, more as the SDK grows) — non-trivial and scheduled for a dedicated cycle. Honest gap acknowledged via the Phase B clone-success Toast.

## Goal

Make a cloned provider's `primaryUrl` (stored in `ClonedProviderEntity.primaryUrl`) actually route the clone's HTTP traffic, instead of routing through the source provider's URLs.

## Current state (post Phase F.1)

`ClonedRoutingTrackerClient` (commit b54e7e63) wraps the source `TrackerClient` and surfaces the clone's descriptor (`id + displayName + baseUrls`). But `getFeature(SearchableTracker)` delegates to the source's feature unchanged — and the source feature's HTTP stack was built around the SOURCE's `baseUrls` at factory.create-time. The clone's `baseUrls` (in the wrapper's descriptor) are visible BUT not read by the actual search path.

User-visible consequence: searching a clone produces results, but they come from the source's URL. The clone-success Toast discloses this: *"Cloned (URL routing pending — searches use source URLs)"*.

## Design options (locked: Option B)

### Option A — `PluginConfig.baseUrlOverride`

Add an `OverridePluginConfig(baseUrl: List<MirrorUrl>): PluginConfig` and modify every tracker factory's `create(config)` to read this and override its internal HTTP client. Each factory must be modified:

- `:core:tracker:ru:RuTrackerClientFactory`
- `:core:tracker:rutor:RuTorClientFactory`
- `:core:tracker:nmclub:NmclubClientFactory`
- `:core:tracker:kinozal:KinozalClientFactory`
- `:core:tracker:archiveorg:ArchiveOrgClientFactory`

- `:core:tracker:gutenberg:GutenbergClientFactory`

Each factory wires its OkHttp/Ktor client with hardcoded base URLs from `descriptor.baseUrls`. To accept an override, the factory must read the config's override field `FIRST`, then fall back to `descriptor.baseUrls`.

**Pros:** uniform contract across factories; the SDK passes the override; each factory wires identically. **Cons:** every factory must be modified; future tracker plugins MUST implement the override or the clone for that tracker breaks silently.

## Option B (LOCKED) — Per-clone MirrorManager + SDK-side URL injection

The SDK already has `LavaMirrorManagerHolder + MirrorManager` per tracker — that's the layer that picks "which baseUrl to actually try" at call time. The Phase F.2 design is:

1. When `LavaTrackerSdk.clientFor(id)` resolves a clone, ALSO build a per-clone `MirrorManager` whose mirror list is the clone's `primaryUrl` + any clone-specific `UserMirrorEntity` rows (the user can add additional mirrors per clone, keyed by `tracker_id = synthetic_clone_id`).
2. `ClonedRoutingTrackerClient` is extended to inject this `MirrorManager` into the source's feature wrappers. Each feature method that issues an HTTP call uses `manager.executeWithFallback { url -> ... }` instead of the source's bound URL.

For this to work, the SOURCE's feature implementations must accept a per-call `baseUrl` parameter OR a `MirrorManager` they can consult. Today they don't — they capture the URL at construction time.

So Option B's refactor is structurally similar to Option A: each tracker plugin's feature impl must be made URL-injectable. The difference is HOW the URL is injected:

- Option A: at `factory.create()` time via `PluginConfig`.
- Option B: at feature-call time via `MirrorManager` parameter.

**Option B is preferred** because it gracefully composes with the existing mirror-fallback machinery (`runWithMirrorFallback`) and with the per-clone `UserMirrorEntity` rows (clone gets its own mirror list, the SDK rotates through them on failure).

**Cons:** larger refactor — every tracker plugin's feature impl gains a `MirrorManager` parameter; the source's existing `MirrorManager` becomes one of many.

## Affected surfaces (cross-module, multi-file)

### Per-tracker plugin (× 6 today)

Each `core/tracker/<name>/src/main/kotlin/lava/tracker/<name>/` directory has feature implementations (`Search.kt`, `Browse.kt`, `Topic.kt`, `Auth.kt`, `Download.kt`). Each MUST accept a `MirrorManager` (or equivalent URL provider) at construction time and use it per-call.

### SDK side

- `LavaTrackerSdk.clientFor(id)` for clones: build a per-clone `MirrorManager` from `ClonedProviderEntity.primaryUrl + UserMirrorDao.observe(syntheticId)`.
- `ClonedRoutingTrackerClient` extended to inject the per-clone manager into each wrapped feature.

### Phase B mirrors section

The `MirrorsSection` in `ProviderConfig` already supports adding `UserMirrorEntity` rows keyed by `tracker_id`. For clones, the `tracker_id` IS the synthetic clone id. No UI change.

## Tests

- Unit tests in each tracker plugin: feature method receives a `MirrorManager`, calls hit the manager's URL.
- New SDK integration test: `LavaTrackerSdkCloneRoutingTest` — wire a clone with `primaryUrl https://rutracker.eu`, fake source's `SearchableTracker` to assert that `feature.search(...)` was called with the clone's URL, not the source's URL.
- Falsifiability rehearsal: revert the SDK seam to use the source's URL — test fails.

## Anti-bluff posture

Per §6.J:

- The test's primary assertion is on the URL the HTTP client tried to hit (`MockWebServer` captures the request URL). Not on mocks or interface calls.
- The clone-success Toast copy is updated to remove the disclosure ("URL routing pending — searches use source URLs") once Phase F.2 ships and end-to-end test passes.

## **Why design only (no implementation in this session)**

Phase F.2 is a cross-module refactor across 6 tracker plugins. Each plugin's feature impls need URL-injection plumbing. Each plugin has its own test suite that must be updated. The cross-cutting scope makes it appropriate for a dedicated session where the refactor's safety can be focused on (existing tests must continue to pass; the new injection seam must not break the source-URL path for non-cloned providers).

## **Phase F.2 acceptance criterion**

The clone-success Toast becomes "Cloned" (no longer "Cloned (URL routing pending — searches use source URLs)"). A clone of RuTracker at `https://rutracker.eu` produces a search-result MockWebServer capture that contains `rutracker.eu` in the request URL, NOT `rutracker.org`.